Exploring Synchronization Tools & Their Applications in Operating Systems

CSE 4501 : Operating Systems

Assigned to :

Saadman Sakib
ID: 210042165

Program : BSc. Engg. SWE
Department of Computer Science and Engineering
Islamic University of Technology

1. About Synchronization

Synchronization ensures that multiple processes or threads access shared resources in a controlled manner to avoid inconsistencies. It's essential for **data integrity, process coordination**, and **resource sharing**.

1.1 Key Definitions:

- **Critical Section**: Code block that accesses shared resources — must be executed by only one process at a time.
- **Deadlock**: A situation where processes wait indefinitely for resources locked by each other.
- **Semaphore**: A synchronization primitive used to control access to a common resource using counters.
- **Monitor**: A high-level abstraction to manage shared resources with mutual exclusion and condition synchronization.

2. Understanding Synchronization Tools

2.1 Semaphores

- **Binary Semaphore** (0 or 1): Acts like a mutex, allows only one process.

- **Counting Semaphore**: Allows multiple processes access, based on the count.

Example:

- Binary: Ensuring one printer access.
- Counting: Managing access to a database connection pool (e.g., max 10 connections).

2.2 Mutexes

How Mutex Differs from Semaphores –-

- Like binary semaphores, but **ownership-based** — only the owner can unlock.

- Often used for **thread-level** synchronization.

Use-case: File writing access among multiple threads.

2.3 Monitors

- Combines **data** + **procedures** + **synchronization** in one module.
- Automatically ensures only one thread accesses monitor procedures at a time.

Example: Java's synchronized methods or `synchronized(this)` blocks.

3. Implementing Synchronization Tools

3.1 C Program: Producer-Consumer with Semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
int buffer[SIZE];
```

```c
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;
    for (int i = 0; i < 10; i++)
    {
        item = rand() % 100;
        sem_wait(&empty);
        // Wait if buffer is full
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        // Signal that buffer is not empty
        sleep(1);
    }
}

void *consumer(void *arg)
{
    int item;
```

```
    for (int i = 0; i < 10; i++)
    {
        sem_wait(&full);
        // Wait if buffer is empty
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        // Signal that buffer is not full
        sleep(2);
    }
}

int main()
{
    pthread_t prod, cons;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
```

```
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

4. Analyzing Concurrency Issues

4.1 How tools help

Issue	Description	How tools help
Deadlock	Circular wait on resources	Avoid nested locks; use timeout mechanisms
Race Condition	Multiple threads read/write shared data simultaneously	Mutexes and semaphores ensure mutual exclusion
Starvation	Some threads never get resource access	Fair semaphores / priority aging mechanisms

4.2 How Issues might still Arise

- **Deadlock:** Two threads acquiring lock1 and lock2 in opposite order.
- **Race Condition:** Two threads incrementing a shared variable without locking.

5. Case Study: Linux vs Windows Synchronization

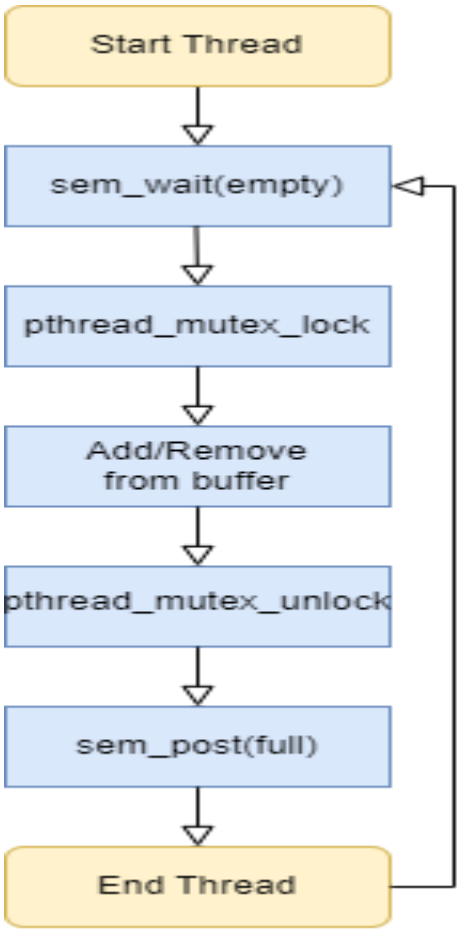
Feature	**Linux**	**Windows**
Semaphores	POSIX semaphores via `sem_t`	Windows Semaphore objects (`CreateSemaphore`)
Mutexes	POSIX mutexes via `pthread_mutex`	Windows Mutex via `CreateMutex`, ownership-based
Monitors	Not natively supported; user-level design	Provided via Critical Sections and Condition Variables

Example :

- Linux uses **futexes** for efficient thread waits.

- Windows uses **IO Completion Ports** and **APC**s for concurrency in services.

6. Visualization : Semaphore in a multi-process environment



7. Conclusion

Synchronization tools like semaphores, mutexes, and monitors are **essential** to manage **safe concurrent execution**. Without them, **critical issues** like race conditions and deadlocks can crash or corrupt systems. Understanding and implementing these tools ensures **system stability, reliability, and performance**, especially in **multi-threaded environments**.